

M A D 3 O O M



THE MAD300M WHITE PAPER

Intelligent Customer Support &
Unified Customer Operations

الورقة البيضاء الأساسية لمتصة مدعوم

PRODUCT VISION • PLATFORM ARCHITECTURE • ROADMAP

October 2026 | Version 1.0

About this document

ITEM	DETAIL
Title	The Mad3oom White Paper — Intelligent Customer Support & Unified Customer Operations
Edition	Foundational edition, version 1.0, October 2026. English and Arabic editions, equivalent in meaning and structure.
Information as of	9 October 2026, from the engineering documentation and source code of the three project repositories.
Intended readers	Prospective partners, technical stakeholders and business audiences
Product stage	Pre-launch. The official launch is still ahead.

Purpose and scope

This paper presents the product vision, platform architecture and roadmap of Mad3oom, an intelligent customer support and customer operations platform. It is written as a credible foundation for conversations with partners and technical stakeholders, and it separates what exists from what is under development, planned or merely envisaged.

How to read the status labels

Every capability in this paper carries one of the labels below. They describe engineering status, not commercial availability.

EXISTING	Implemented in the project repositories, with automated tests where noted. It does not imply production maturity, wide use or independent validation.
IN DEVELOPMENT	Active engineering work: code written or partly delivered, not yet complete or enabled for users.
PLANNED	Designed and intended, with no delivered implementation yet. Subject to validation and priorities.
FUTURE	A long-term opportunity or exploratory direction. No commitment is made.
RECOMMENDATION	Architectural or security guidance proposed by this paper or by internal reviews. It is advice, not a statement about the current system.

NOTICE

Statements about the future are intentions that depend on technical validation and business priorities. They are not commitments, and no delivery dates are given. This paper reports no customer counts, revenue, market share, security certifications, partnerships or measured real-world accuracy, because none are documented. See [Appendix C](#) for the evidence basis.

Contents

Chapter 1	Executive Summary	5
1.1	What Mad3oom is	5
1.2	Whom it serves	5
1.3	The problems it addresses	6
1.4	Main components	6
1.5	The value of one connected workflow	7
1.6	Where the project stands	7
Chapter 2	Vision and Problem Statement	8
2.1	The vision	8
2.2	Six problems in customer support	8
2.3	What the problems imply for design	9
Chapter 3	The Mad3oom Solution	10
3.1	A unified approach	10
3.2	How the parts are meant to relate	11
3.3	Two kinds of work, kept separate	12
Chapter 4	Mad3oom Workspace	13
4.1	The idea	13
4.2	What exists and what does not	13
4.3	How it is built	14
4.4	Integrity and access rules	15
4.5	Engineering evidence and limits	15
Chapter 5	Mad3oom Relay	16
5.1	The business problem	16
5.2	What a continuity record is	16
5.3	How Relay complements ticketing	17
5.4	Capabilities against the intended concepts	18
5.5	Governance of evidence	19
5.6	Stage and limits	20
Chapter 6	SIE: Support Intelligence Engine	21
6.1	What SIE is, and what it is not	21
6.2	Interpreting a problem versus coordinating the work	21
6.3	A layered design	22
6.4	Why the layers matter	23
6.5	What has been verified, and what has not	24
6.6	Where SIE runs	24
Chapter 7	Integrations and Connectivity	25
7.1	Integration principles	25
7.2	Channels and connected systems	26
7.3	WhatsApp Cloud API: three different things	27
7.4	What is not claimed	28

Chapter 8	Technical Architecture	29
8.1	Verified technical context	29
8.2	Conceptual architecture	30
8.3	Principles at work	31
8.4	Recommendations and future directions	31
Chapter 9	Security, Privacy and Governance	33
9.1	Principles, safeguards and gaps	33
9.2	Governance and assurance	34
Chapter 10	Product Differentiation and Value	36
10.1	Five design principles	36
10.2	How the parts combine: an illustrative journey	37
10.3	Measures that would show the value	37
10.4	What is not claimed	37
Chapter 11	Product Roadmap	38
11.1	Existing foundations	39
11.2	Current development and refinement	39
11.3	Planned capabilities	40
11.4	Long-term opportunities	40
Chapter 12	Conclusion	41
Appendix A	Capability Status Register	42
Appendix B	Glossary	44
Appendix C	Basis of Preparation and Limitations	46
C.1	Sources reviewed	46
C.2	Method	46
C.3	What was not verified	47
C.4	Editorial rules	47
C.5	Document history	47

01

CHAPTER 1

Executive Summary

Mad3oom is a platform for technical support and customer operations. It brings tickets, customer conversations, customer records, knowledge and follow-up work into one structured environment, and adds a layer of support-specific intelligence on top.

AT A GLANCE

- Mad3oom unifies tickets, conversations, customer records, knowledge and operational follow-up in one platform.
- Three layers shape its design: **Workspace** for multi-context work, **Relay** for continuity and accountability, and **SIE** for support intelligence.
- Foundations exist in code; the official launch is still ahead. Every capability in this paper is labeled by stage.
- The architecture favors modularity, clear interfaces and explainable decisions over feature breadth.

1.1 What Mad3oom is

Mad3oom is a software platform for support teams. It lets a team receive customer requests, understand them, assign responsibility, keep the context of every interaction and follow each request until it is resolved. It is designed as a coherent technology product with a defined architecture, rather than as a chat widget or a conventional ticket list.

The design is Arabic-first: interfaces are right-to-left, and the intelligence layer is built to handle Arabic dialects and Arabic written in Latin letters, while the engine can also reply in English.

1.2 Whom it serves

- **Support agents and supervisors**, who need one place to work on tickets and conversations without losing context.
- **Companies** that support their own customers and need structure, clear ownership and visibility over open work.
- **Customers**, who reach support through a portal and messaging channels and should not have to repeat themselves.
- **Platform operators**, who need governance over access, data and automation.

1.3 The problems it addresses

Support work degrades when customer information is scattered, context is lost between tools, follow-ups are forgotten, handoffs are incomplete, ownership is unclear and intelligence is applied inconsistently. These are common industry problems, analyzed in [Chapter 2](#). They are described here as the reasoning behind the design, not as findings of a completed customer study.

1.4 Main components

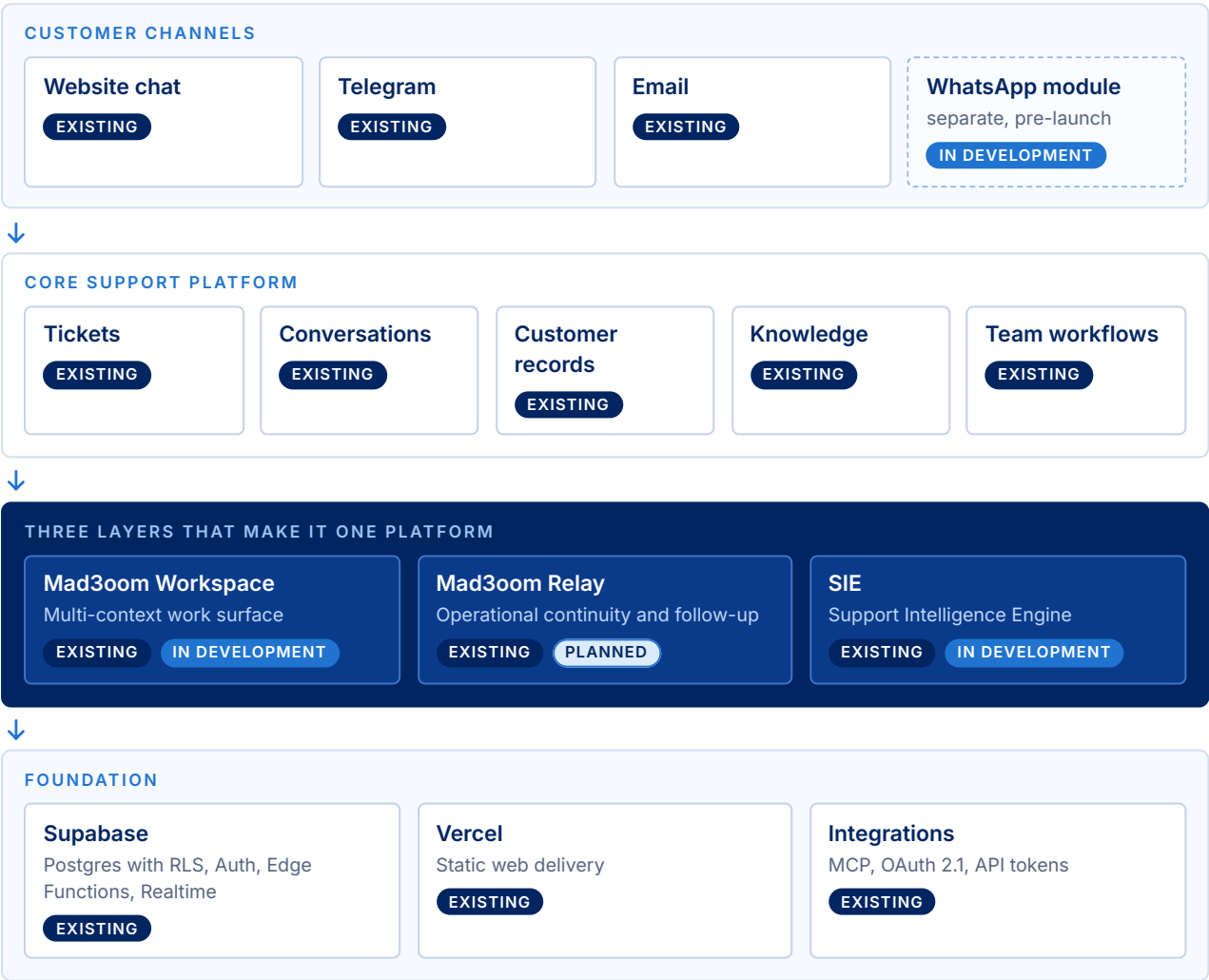


Figure 1. Mad3oom at a glance. A conceptual map of the platform, with the stage of each part. It is not a deployment diagram.

Table 1. Components and their stage. Workspace, Relay and SIE each combine delivered parts with parts still in development or planned; the detail is in [Chapter 4](#), [Chapter 5](#) and [Chapter 6](#).

COMPONENT	ROLE	STAGE
Core support platform	Tickets with SLA targets and notifications, a helpdesk inbox with human–bot handoff, customer history, a knowledge base and team workflows.	EXISTING
Mad3oom Workspace	A tabbed, split-panel surface that hosts tickets, conversations and customer records side by side.	EXISTING IN DEVELOPMENT
Mad3oom Relay	Continuity records that keep evidence, an accountable owner, a next action and a deadline together.	EXISTING PLANNED
SIE	A deterministic engine that interprets customer messages and decides the next support action.	EXISTING IN DEVELOPMENT
Integration layer	Channel adapters, MCP with OAuth 2.1, API tokens and event-driven side effects.	EXISTING

1.5 The value of one connected workflow

The intended value comes from three properties that separate tools rarely provide together.

- **Context travels with the work.** A ticket, the conversation behind it and the customer's history are reachable from the same place.
- **Responsibility is explicit.** Each piece of follow-up work has one accountable owner, a next action and a deadline that is visible to the team.
- **Intelligence is controlled and explainable.** The engine chooses from a closed set of actions through ordered rules, and records why.

These are design intentions. Whether they translate into fewer missed requests or faster resolution can only be shown by measurement in real use, and this paper reports no such measurements.

1.6 Where the project stands

STAGE OF THE PROJECT

Mad3oom has not launched. The paper describes foundations that exist in the code, work that is under way and directions that are planned. Internal engineering reviews in October 2026 identify priority hardening work to finish before open commercial availability and favor a controlled pilot as the first step. No customer deployments, benchmarks of real-world accuracy or security certifications are claimed.

02

CHAPTER 2

Vision and Problem Statement

The vision is simple to state and demanding to deliver: technical support that is organized, context-aware, accountable and intelligent. The obstacles are not unique to Mad3oom. They are the recurring failure points of support operations assembled from disconnected tools.

2.1 The vision

Mad3oom aims to make technical support more organized, more context-aware, more accountable and more intelligent. The order is deliberate and reflects design logic, not a delivery schedule. Organization comes first, because intelligence applied to disorganized work produces confident mistakes. Context follows, because every later improvement depends on knowing what has already happened. Accountability comes next, because work without an owner is the main source of dropped requests. Intelligence is the last layer.

2.2 Six problems in customer support

NATURE OF THIS ANALYSIS

These are recurring industry problems that the platform is designed to address. They are drawn from general support-operations practice and from the design reasoning in Mad3oom's engineering documents. They are not quantified findings from a completed customer study, and this paper reports no statistics about them.

1. Customer information is scattered

One customer can exist as a chat session, a ticket, a profile and an email thread, each in a different screen or product. Agents rebuild the picture by hand, and different agents rebuild it differently.

2. Context is lost when switching

Moving from a conversation to a ticket to the customer's history usually means leaving one view to open another, and half-written replies and the reasoning behind a decision are left behind.

3. Follow-ups are forgotten

A commitment made in a conversation, such as "I will check and come back tomorrow", often lives only in the message itself. Unless someone copies it into a task list, it depends on memory, and unresolved requests quietly age.

4. Handoffs are incomplete

When responsibility moves between people or shifts, the receiving agent often has the case but not the context: what was promised, what was already tried and what is still unknown.

5. Ownership, status and next action are hard to see

Supervisors can see that work is open, but not always who owns it, what the next step is or whether a deadline has passed. Work with no owner is invisible until a customer complains.

6. Intelligence is hard to apply consistently

Automation and AI features added tool by tool behave differently across channels, are difficult to explain afterwards and hard to test. Without a controlled design, intelligence amplifies inconsistency instead of reducing it.

2.3 What the problems imply for design

Each problem points to a design requirement. The table shows the response chosen for each one, the component that carries it and the stage that response has reached.

Table 2. Problems, design responses and stage. Where two stages appear, the first covers what exists and the second what is planned.

PROBLEM	DESIGN RESPONSE	CARRIED BY	STAGE
Scattered information	Tickets, conversations and customer records in one platform, shown side by side.	Core platform, Workspace	EXISTING
Lost context	Several work contexts open at once; unsent text is protected when tabs move or close.	Workspace	EXISTING
Forgotten follow-ups	Records with a next action, a deadline and a derived overdue flag. Scheduled reminders and escalation are planned.	Relay	EXISTING PLANNED
Incomplete handoffs	Controlled reassignment with a history of changes. Handovers that the receiver must accept, clarify or decline are planned.	Relay	EXISTING PLANNED
Limited visibility	Lists filtered by owner, status and category, with unassigned work flagged. A continuity monitor that explains why a record needs attention is planned.	Relay	EXISTING PLANNED
Inconsistent intelligence	One layered, deterministic engine behind every channel, with a recorded reason for each decision.	SIE	EXISTING IN DEVELOPMENT

These responses explain the structure of the platform described in [Chapter 3](#).

03

CHAPTER 3

The Mad3oom Solution

Mad3oom's answer is not another silo. It is a connected set of parts: the records of support work, a surface to work on them, a layer that keeps work from being dropped, and an engine that understands customer messages.

3.1 A unified approach

The platform treats six things as parts of one workflow rather than as separate tools. Each is described below as it exists today in the project's repositories.

Table 3. The six elements of the unified approach and the stage each has reached.

ELEMENT	WHAT IT PROVIDES	STAGE
Support tickets	A ticket lifecycle from opening to resolution, including customer confirmation, rejection or reopening of a resolution; SLA targets; assignment and distribution; notifications in the platform, by email and through Telegram.	EXISTING
Customer conversations	Website chat and Telegram feed a helpdesk inbox with teams, assignment, tags, internal notes, attachments, scheduled replies and an explicit hand-off between the bot and a human agent.	EXISTING
Customer records	A customer history page that gathers a customer's tickets and conversations in one view.	EXISTING
Knowledge resources	A public knowledge base and help center, plus knowledge entries that the intelligence engine can attach to answers.	EXISTING
Team workflows	Roles for platform staff and companies, an automation builder for rules and actions, and company accounts with their own dashboards.	EXISTING
Operational follow-up	Mad3oom Relay: continuity records with an owner, a next action, a deadline and source evidence. See Chapter 5 .	EXISTING PLANNED

3.2 How the parts are meant to relate

Three layers sit on top of the core records. **Workspace** is the surface on which agents see those records together. **Relay** is the layer that keeps follow-up work attached to its evidence, its owner and its deadline. **SIE** is the engine that interprets what customers write. The figure below shows the intended relationships, and the table that follows states which of them are implemented today.

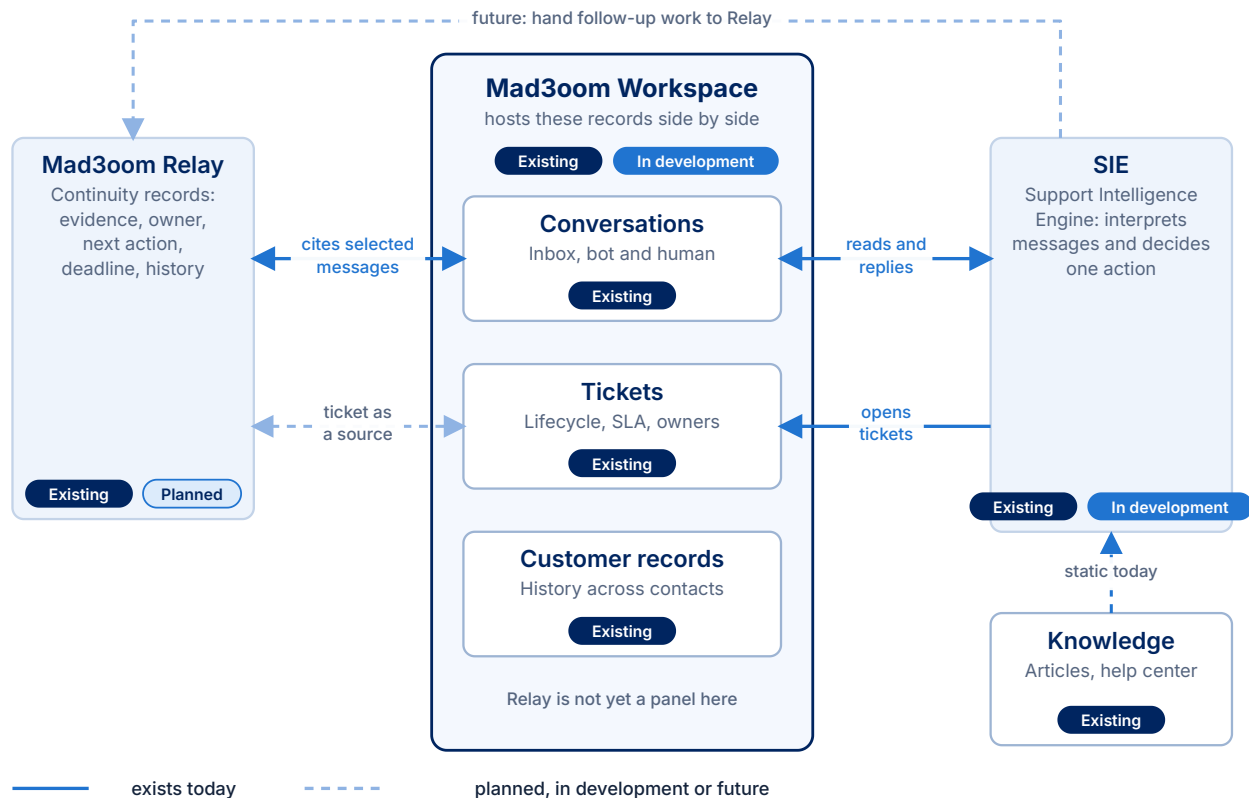


Figure 2. Intended relationships between the parts. Solid lines exist today; dashed lines are planned, in development or future. This is a conceptual map, not a data-flow specification.

Table 4. Relationships between the parts and their stage.

RELATIONSHIP	WHAT IT MEANS	STAGE
Workspace and the hosted pages	The existing pages are hosted as panels in an embed mode; they keep their own logic and access rules.	EXISTING
Relay and conversations	An agent selects messages in a conversation and creates a continuity record that cites them.	EXISTING
Relay and tickets	A ticket as a source of a continuity record.	PLANNED
Relay inside Workspace	A Relay record opened as a panel beside the conversation it came from.	PLANNED
SIE and conversations	The engine reads inbound website and Telegram messages and writes its replies; a human takeover stops it.	EXISTING
SIE and tickets	When the decision is to open a ticket, the message, state and ticket are committed together.	EXISTING
Knowledge and SIE	Static knowledge entries are attached to answers. Grounding in live account data and published knowledge is in development.	EXISTING IN DEVELOPMENT
SIE and Relay	The engine handing a follow-up to Relay. No such link exists today.	FUTURE

LIMITATIONS

Not every integration or workflow described in this paper is fully implemented. Where a relationship is marked as planned, in development or future, the platform does not yet behave that way. The rest of the paper keeps the same distinction.

3.3 Two kinds of work, kept separate

A central design decision is to keep **interpretation** apart from **coordination**. Interpretation answers the question “what is this customer telling us, and what should happen next in this conversation?”. That is SIE's job. Coordination answers “who is responsible for what, by when, and with which evidence?”. That is Relay's job, together with ticketing. Keeping them apart lets each be built, tested and improved on its own, and prevents an intelligent component from silently taking decisions that belong to people.

04

CHAPTER 4

Mad300m Workspace

Support work rarely fits on one screen. Mad300m Workspace lets an agent keep several tickets, conversations and customer records open at once, arranged as tabs and split panels, without losing what has been typed or where they left off.

4.1 The idea

Workspace is a multi-tab work surface inside the Mad300m administration console. Rather than treating the inbox, the tickets list and the customer history as separate pages that replace one another, it hosts them side by side. An agent can read a conversation, open the related ticket beside it and look at the customer's history in a third pane. Each stays as it was left.

4.2 What exists and what does not

Table 5. Workspace capabilities by stage. A row with no capability name continues the row above.

CAPABILITY	DETAIL	STAGE
Tickets, conversations and customers in one place	Six panel types: the inbox, tickets and customer-history pages, and single-conversation, single-ticket and single-customer panels.	EXISTING
Switching without losing context	Hosted pages keep running while hidden. Closing a tab that holds unsent text asks for confirmation; moving or docking a tab never reloads it.	EXISTING
Organizing views	Tabs, groups and nested splits made by dragging a tab to any edge or the center; resizable dividers; quick open with Ctrl+K or Cmd+K; starter layouts; reopening the last closed tab.	EXISTING
Opening related records beside the source	Following a link from a ticket to its customer opens the customer in another group, or splits the view if only one group exists.	EXISTING
Layouts that survive a reload	Saved per device in browser storage, with titles resolved live and never stored.	EXISTING
	Saving each agent's layout on the server, so it follows the agent across devices. The migration is written and tested locally; it is not applied to production.	IN DEVELOPMENT
Keyboard, RTL and small screens	Tab-list semantics, direction-aware arrow keys, menu alternatives to dragging, mirrored layout in Arabic, and a one-panel compact mode below 900 px.	EXISTING

CAPABILITY	DETAIL	STAGE
Relay as a panel	Opening Relay records as a first-class panel type next to the conversation they cite.	PLANNED
More panel types	Knowledge and reporting panels. The registry is designed to accept new types without changes to the layout engine.	PLANNED
Adaptive organization	Named layouts for recurring tasks and context shared across panels beyond refresh hints.	FUTURE

4.3 How it is built

The inbox and tickets pages own global state and fixed element identifiers, so they cannot be mounted twice in one page. Rewriting roughly 3,600 lines of tested helpdesk code was ruled out. Workspace therefore **hosts the existing pages** in same-origin frames running in an embed mode, which removes the surrounding chrome and narrows each page to one record. A tab behaves like a browser tab of that page, with the page's own logic, access checks and live updates intact.

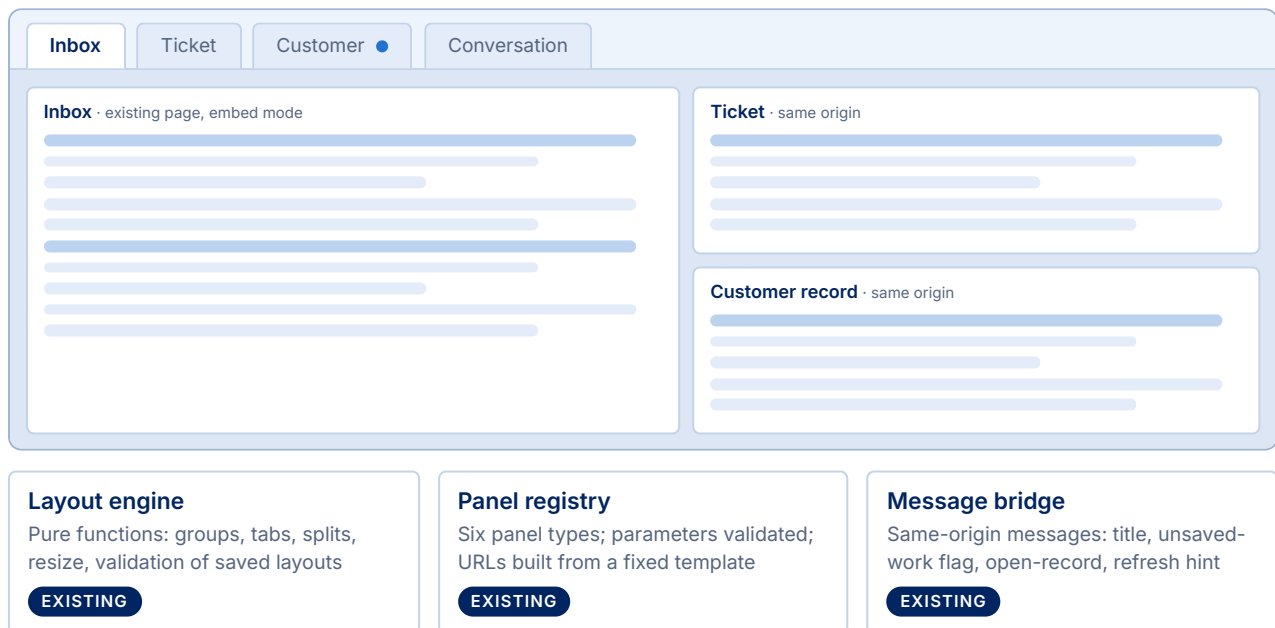


Figure 3. The Workspace model. A shell arranges tabs and split groups; each panel is an existing page in embed mode. The three supporting parts are shown below the window. The window is illustrative, not a screenshot.

- **Positioned frames.** Frames are placed over the area of the group that shows them rather than moved in the page structure, because moving a frame would reload it and discard an unsent reply.
- **A pure layout model.** Tabs, groups and splits are described by plain data and pure functions, tested without a browser. Edges are logical (start and end), so right-to-left is correct by construction. Limits: 40 panels, 12 groups, depth 6.
- **A panel registry.** An allow-list of panel types with validated parameters. Addresses are built from fixed templates, so a restored or messaged panel cannot inject other parameters.

- **A message bridge.** Hosted pages report their title, whether they hold unsent work, and requests to open a related record. The shell accepts a message only from the same origin and only from a frame it created.

4.4 Integrity and access rules

These rules are enforced in code and covered by the repository's tests.

- Moving, docking or splitting a tab never reloads or recreates its frame.
- Closing a tab, closing others, resetting the layout or leaving the page checks for unsent work first. The cap on live frames never unloads a panel that holds unsent work.
- Workspace never calls a write operation itself. Every write happens inside the hosted page through its existing routes, so the shell cannot duplicate a submission.
- Saved layouts are never trusted. Structure is validated, and each record panel is checked against the user's own access before its frame is created; a record that is no longer visible appears as an unavailable tab.
- A saved layout contains only panel types and record identifiers, never names, email addresses or message text.
- If saving fails, Workspace keeps working with the in-memory layout, and every page remains usable at its own address.

4.5 Engineering evidence and limits

Table 6. Development-environment checks. They show that the design works as specified; they say nothing about how Workspace performs for agents on production data.

CHECK (REPOSITORY, 9 OCTOBER 2026)	RESULT
Layout engine unit tests	28 of 28 pass
Chromium render tests of the real page code on test doubles	21 of 21 pass
Size of the whole workspace	about 46 KB compressed, 13 files, no new dependency
Layout operations at the limit (40 panels)	under 1 ms each in Node
Re-render on tab switch, typical layout (6 tabs)	median about 9 ms in headless Chromium

LIMITATIONS

Each live panel is a full page instance with its own real-time connection, so the number of live panels is capped at six and the least recently shown clean panel is unloaded. A draft typed in the inbox does not follow a conversation opened as its own tab. Hosted pages keep their existing language while the shell follows the language switch. Drag-and-drop is not supported on touch screens, where compact mode is used instead. No automated accessibility audit has been run. Server-side layout saving is not applied to production, so layouts are per browser today.

05

CHAPTER 5

Mad3oom Relay

Tickets record cases. Relay records commitments. It is the follow-up and coordination layer that keeps a piece of work attached to its evidence, its owner and its deadline, so that a promise made in a conversation does not depend on someone remembering it.

5.1 The business problem

Work loses its context, its owner or its next action when responsibility moves between people or shifts. The usual symptoms are forgotten follow-ups, problems that outlive a shift, handovers without context, commitments discussed in chat and never tracked, and customers who must explain their issue again. A ticketing system answers whether a case is open. It does not by itself answer who has promised what, by when, on the strength of which evidence.

5.2 What a continuity record is

DEFINITION

A **continuity record** is the operational unit of Relay. It combines evidence (references to selected messages), one accountable owner or an explicit "unassigned", a next action, a deadline with an explicit time zone, a completion that must be justified, and an append-only history.

Today a record is either a **follow-up**, which must have a next action and a deadline, or an **issue**, a problem that needs resolving. A third kind, **handover**, exists in the data model but is switched off (see below). Each record can also carry one of nine categories, such as order status, payment and billing, technical issue or complaint.

5.3 How Relay complements ticketing

Table 7. Ticketing and Relay answer different questions and are designed to be used together.

	TICKET	RELAY RECORD
Main question	Is this customer case open, and how is it progressing?	Who committed to what, by when, on which evidence?
Scope	A customer-facing case.	Any piece of follow-up work, whether or not a ticket exists.
Evidence	The messages of the ticket thread.	Selected messages cited from conversations; access to each excerpt is re-checked on every read.
Time	SLA targets by priority.	An explicit deadline and time zone. Overdue is derived and never resolves anything by itself.
Ownership	An assigned agent or team.	One accountable owner or an explicit "unassigned", with rules on who may assign.
History	Ticket activity.	An append-only event log that holds identifiers and changes, not message text.

5.4 Capabilities against the intended concepts

Table 8. Relay capabilities against the six intended concepts. A row with no concept name continues the row above.

CONCEPT	WHAT EXISTS OR IS INTENDED	STAGE
Tracking follow-ups and outstanding actions	Follow-up and issue records with a lifecycle (open, scheduled, in progress, waiting, resolved, cancelled), a next action, a priority and a category; a filterable list; creation from selected inbox messages through a three-step dialog, or by hand.	EXISTING
Preserving context through summaries	Each record holds a summary written by the person who creates it and cites its source messages. Excerpts are shown only to people who still have access to the original conversation. Automatic summarization is not implemented.	EXISTING
	Assisted extraction of titles, dates and next actions, with every suggestion editable and never confirmed automatically.	FUTURE
Clarifying ownership	One owner field; unassigned work is flagged. Only supervisors and staff explicitly granted the assign privilege may give work to someone else; an owner without it can only take a record or release it. The privilege is granted explicitly by an authorized administrator.	EXISTING
Supporting handoffs	Reassignment under the rules above, with a history of every change.	EXISTING
	A formal handover that the receiver must accept, query or decline; handovers to a team where the first to accept wins; end-of-shift bundles. The record keeps the previous owner until acceptance. The data model reserves this, and it is switched off.	PLANNED
Making pending and overdue work visible	An overdue flag derived from the deadline, shown in lists; filters by status, owner and category.	EXISTING
	Reminders before a deadline, escalation, and a continuity monitor that lists records needing attention with a reason (unassigned, no next action, overdue, handover pending, owner inactive).	PLANNED
Continuity and accountability	An append-only event log; version checks that reject stale edits; idempotency keys that prevent duplicate records on retries.	EXISTING

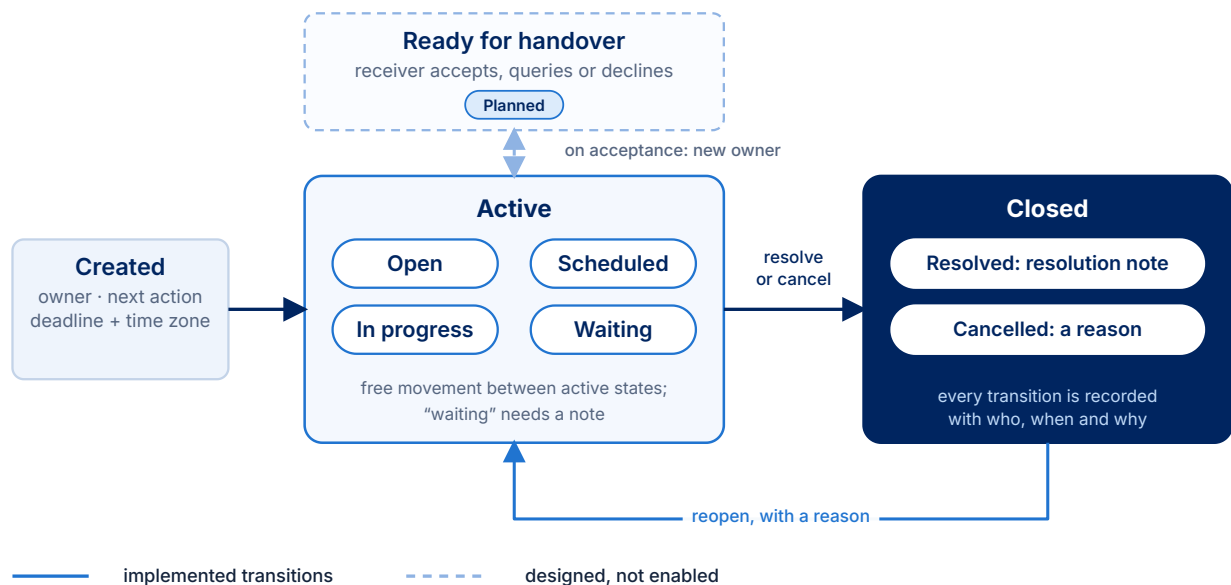


Figure 4. The lifecycle of a continuity record. Solid arrows are implemented transitions; the dashed branch is the designed but disabled handover.

5.5 Governance of evidence

Relay copies a small piece of customer conversation into another place, so its design treats that copy carefully. These rules are implemented in the database and covered by its SQL tests.

- **Minimal capture.** Only the messages the user selects are stored, up to 4,000 characters per source.
- **Access follows the conversation.** An excerpt is shown only to someone who has access to the record and who still has access to the original conversation. The check runs on every read and fails closed.
- **Retention and redaction.** Excerpts are hidden 365 days after a record is closed and then physically redacted by a daily job. Redaction cannot be undone, and a customer's data-deletion request is handled by a supervisor redaction.
- **Sensitive content.** Content that looks like a card number, a one-time code, a password or a national identifier needs an explicit acknowledgement before it is saved.
- **History without content.** Audit events hold identifiers, state changes and hashes, never excerpt text or free-text values, so content can be removed without breaking the history.
- **Trash and erase.** A removed source goes to a trash and can be restored; permanent erase belongs to the platform owner alone. This is written and tested locally and is not applied to production.

5.6 Stage and limits

Table 9. Relay by delivery phase.

PHASE	CONTENT	STAGE
Core persistence and security	Records, sources, snapshots, events, ownership rules, retention. The engineering record states it is applied to the production database and enabled for platform staff.	EXISTING
Native selection and conversion	The inbox dialog, type preview, the Relay page and assignment privileges.	EXISTING
Trash and owner control	Removal with restore; owner-only erase and privilege grants.	IN DEVELOPMENT
Handover	Handover records with acceptance, clarification and decline; end-of-shift bundles.	PLANNED
Scheduling and monitor	Reminders, escalation, failed-job visibility and the continuity monitor.	PLANNED
External API and browser capture	A token-based API for other clients, and a browser extension that records a selected passage with its page address.	PLANNED
Assisted extraction; company workspaces	Suggested fields from messages; Relay for companies' own support teams.	FUTURE

LIMITATIONS

Relay currently works only in the platform's own support inbox, for platform staff; company workspaces are refused by design for now. It is not yet linked to tickets, to Workspace panels or to SIE. It sends no reminders, and it produces no automatic summaries. The measures intended to show its value (records created per source, share of active records with an owner and a next action, overdue count and age, handover acceptance time, reminders delivered against failed) are defined but not yet reported, so this paper makes no claim about how much it reduces missed requests.

06

CHAPTER 6

SIE: Support Intelligence Engine

SIE is Mad3oom's specialized engine for technical-support conversations. It reads what a customer writes, weighs it against a catalog of known support situations, and chooses one next action from a closed set while recording why. It is deliberately not a general-purpose chatbot.

6.1 What SIE is, and what it is not

SIE is a deterministic, rule- and data-driven engine written as dependency-free JavaScript modules. For the same input and the same configuration it produces the same decision: time is injected rather than read, there is no randomness, and ties are settled by an explicit rule. Its decision is one action from a closed vocabulary, chosen by ordered rules, and each rule records whether it matched and why.

Determinism and a closed action set make the engine explainable and testable. The trade-off is coverage: SIE handles the situations its catalog describes and does not improvise outside it.

WHAT IS NOT CLAIMED

This paper does not claim human-level reasoning, autonomous resolution of every issue, guaranteed accuracy or independently verified production performance for SIE. A generative model does not make SIE's decisions today. Whether controlled generative assistance could be added is a long-term direction, discussed in [Chapter 11](#).

6.2 Interpreting a problem versus coordinating the work

SIE answers one question: what is this customer telling us, and what should happen next in this conversation? Its possible outputs are conversational: answer from knowledge, ask for more detail, check an understanding with the customer, open a ticket, or hand the conversation to a person. It does not assign owners, set deadlines or track commitments. That coordination belongs to ticketing and to Relay ([Chapter 5](#)).

Today SIE's only operational effects are its chat reply and, when it so decides, a ticket created in the same atomic step. A human takeover of a conversation stops it from replying. There is no link from SIE to Relay yet; handing follow-up work from the engine to Relay is a future possibility.

6.3 A layered design

SIE is organized in nine layers plus a trust boundary that cuts across them. The figure lists them in runtime order, which differs from their numbering: knowledge is attached before the reply is written.

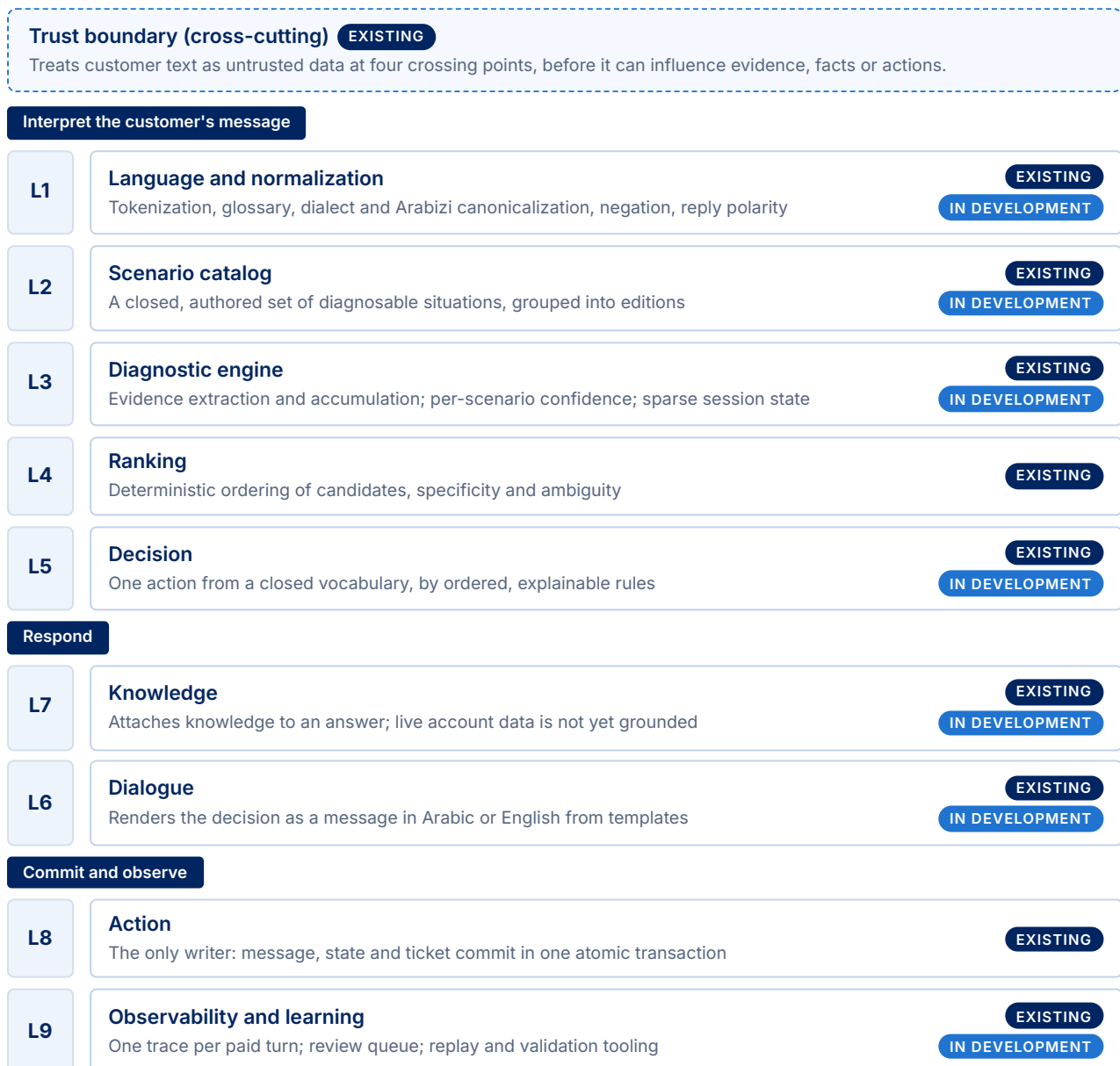


Figure 5. SIE layers in runtime order, with stage. The numbering is the layers' own; the trust boundary applies at four points where customer text could affect evidence, facts or actions.

Table 10. The seven responsibilities of the engine and, for each, what exists and what remains in development.

RESPONSIBILITY	WHAT EXISTS	STILL IN DEVELOPMENT
Language understanding and normalization	Tokenization; a technical glossary that maps many phrasings to canonical terms; Arabic-dialect normalization; Arabic written in Latin letters; reply language. Matching respects whole-word boundaries, negation and yes/no polarity.	Conversational detectors in the orchestrator still re-read raw text and are to be moved onto the language layer's output.

RESPONSIBILITY	WHAT EXISTS	STILL IN DEVELOPMENT
Scenario-based reasoning and interpretation	A closed catalog of authored scenarios, each with a signature of expected evidence, resolutions and questions; grouped into editions and extensible by a published overlay. About 900 scenarios in the largest edition at the October 2026 audit.	About one fifth of the catalog are conversational scenarios, not problems; they are to be tagged and excluded from problem matching.
Diagnostic analysis	Extracts evidence from each message, accumulates it over the conversation and scores each candidate scenario by how much of its signature is present. Session state can be stored in a compact sparse form.	Evidence that can contradict or fade, and strict separation between conversations.
Candidate ranking and decision logic	Deterministic ordering with explicit tie-breaks, specificity and an ambiguity flag; then ordered decision rules that choose one action and record every rule evaluated.	The decision engine is to own every route, with expiring pending prompts and a ticket state that records only committed facts.
Dialogue management	Decisions are rendered as messages in Arabic or English from templates, with a parity test between the two languages.	All customer-visible text to come from one template source, with checks that a message never contradicts the decision.
Knowledge retrieval and application	Static knowledge entries attach to answers for a small set of scenarios. A separate inverted-index step narrows scoring to relevant candidates and is tested to give the same results as a full scan.	Grounding answers in live account data (ticket and subscription status) and in published knowledge, before the decision is made.
Action-oriented support workflows	The action layer is the only writer. One decision becomes one atomic transaction covering the message, the session state and, when decided, the ticket. Each paid turn is traced once, with the decision's intent and the executed outcome kept apart.	Charging quota only for a committed reply, attachment metadata, and a database-enforced publish gate.

6.4 Why the layers matter

- **Modularity.** Dependencies run one way: an import-graph review established that the engine imports nothing from channel or platform code, and a test enforces the same boundary on the channel side. Scenarios and glossary are data behind provider interfaces, not code.
- **Testability.** Layers are pure functions with an injected clock, so the suite runs without a database or network. At the time of writing all 1,264 tests in the engine repository pass. A guarantee registry lists 66 behavioral guarantees; a continuous-integration check fails if a guarantee marked enforced has no passing evidence.
- **Maintainability.** Each layer has its own tests and a narrow contract, there is a single writer, and every decision carries the rules that produced it, so a surprising reply can be traced to a cause.
- **Controlled evolution.** Behavioral changes ship behind settings flags that default to current behavior, deployments pin an exact engine commit, and fixes proceed in work packages with stated exit criteria.

NOTE

A layered design does not by itself make behavior correct. The October 2026 audit found that a large share of behavior lived in an orchestration layer outside the numbered layers, and the remediation program is moving it back inside them.

6.5 What has been verified, and what has not

Table 11. The evidence behind SIE, stated with its limits.

EVIDENCE	WHAT IT SHOWS
Automated tests	1,264 tests pass in the engine repository (run on 9 October 2026). They verify specified behavior under controlled conditions. They are not accuracy measurements.
Behavioral guarantees	66 registered; 24 are enforced by passing tests and 42 are pending, each tied to a named work package.
Internal end-to-end audit, 6 October 2026	Realistic synthetic conversations were replayed through the real engine under production settings. The audit found defect classes in the orchestration layer, ticket state, evidence handling, knowledge grounding and tracing, and led to the remediation program.
Remediation program	Test foundation, truthful tracing and language primitives are delivered. Decision ownership, belief that can fall, grounding, attachments, metering and dialogue consolidation follow. Where risk was found, interim mitigations were applied by configuration.
Scale benchmarks, synthetic	On generated catalogs, computation time stayed in milliseconds up to 10,000 scenarios; resident memory and index build time, not computation, limit larger sizes. These are synthetic measurements, not production load results.
Trust boundary	On a synthetic corpus it caught 18 of 18 hostile messages and escalated none of 338 legitimate ones. The false-positive rate on real traffic is unmeasured.
Accuracy, resolution rate, satisfaction	Not measured. No such figure is claimed.

6.6 Where SIE runs

SIE is deployed behind the website chat and the Telegram channel as Supabase Edge Functions, each pinned to an exact engine commit. Channels reach it through a shared adapter layer ([Chapter 7](#)). It is offered in editions that differ in scenario coverage, and administrators set its behavior through a settings schema. Its operation is pre-launch and at limited volume.

07

CHAPTER 7

Integrations and Connectivity

Mad3oom's integration vision is that support should meet customers where they already are and connect to the systems teams already use, without letting any single vendor's details leak into the core platform.

7.1 Integration principles

- **Adapters isolate vendors.** Each messaging channel is an adapter with four duties: verify the request, parse the vendor format into one standard message, send replies, and optionally show typing. The engine knows nothing about channels, and a test that reads the source fails if vendor details leak out of the adapter folders.
- **Identity before spend.** A messaging account must be linked to a platform account, through a single-use code that expires after 15 minutes, before the engine replies. A stranger who finds the bot cannot consume a paying customer's allowance.
- **Verify and de-duplicate before cost.** Unauthenticated requests are rejected first, and duplicate deliveries are detected before anything is charged.
- **Open, standard interfaces.** OAuth 2.1, the Model Context Protocol and token-based APIs, rather than private conventions.

7.2 Channels and connected systems

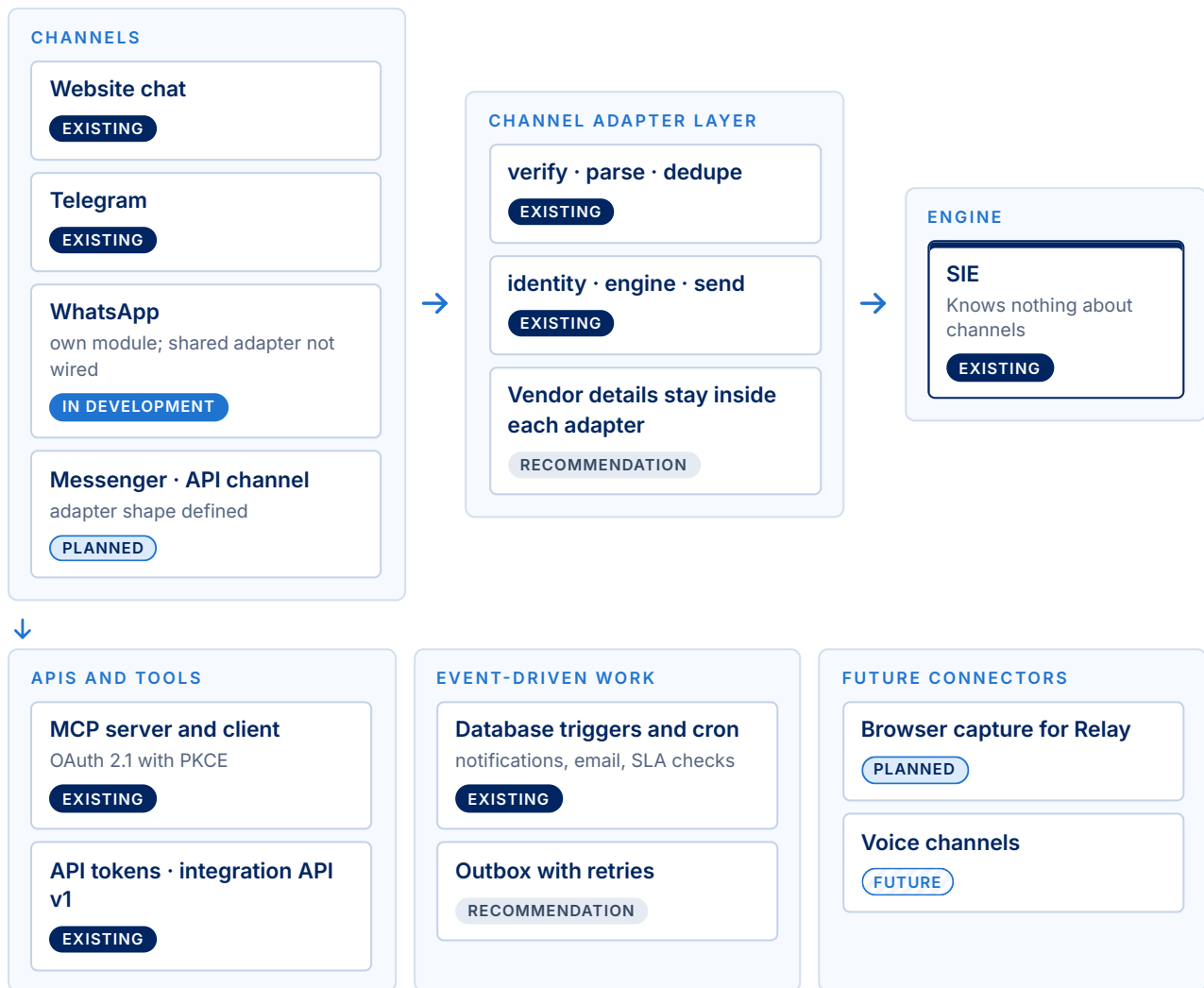


Figure 6. Connectivity map. Channels reach the engine through one adapter layer. The lower row shows APIs and tools, event-driven work and future connectors. It is conceptual.

Table 12. Integrations by stage. A row with no name continues the row above.

INTEGRATION	WHAT EXISTS OR IS INTENDED	STAGE
Website chat and customer portal	A chat widget that runs in the customer's browser under their own session and calls the engine; customer and company dashboards.	EXISTING
Telegram	A support channel handled by the engine, with webhook secret verification and account linking; also notifications and one-time codes for staff and customers.	EXISTING
Email	Ticket emails sent through an email provider, and an inbound-mail webhook that receives replies.	EXISTING
WhatsApp	A separate WhatsApp module. See the next section; it is not part of the initial launch scope.	IN DEVELOPMENT
Messenger and an API channel	Adapter shapes are defined in the engine's channel layer; no transport is connected.	PLANNED

INTEGRATION	WHAT EXISTS OR IS INTENDED	STAGE
MCP server and client; OAuth 2.1	Mad3oom can expose tools to MCP-compatible AI clients and connect to external MCP servers. Its authorization server uses mandatory PKCE, scoped tokens stored hashed, rotating refresh tokens and a page where users revoke connected apps.	EXISTING
API tokens and the integration API	Users can issue scoped API tokens. A versioned integration API lets an approved external application request a pre-approved template message without knowing the provider, tokens or database; its keys are stored hashed, distinguish test from live and can be rotated or revoked.	EXISTING
Event-driven work	Database triggers and scheduled jobs produce notifications, email, SLA checks, retention jobs and scheduled replies; an automation builder runs rules when tickets are created; outgoing webhooks notify other systems.	EXISTING
	Delivery through an outbox with retries and dead-letter handling, so that a failed side effect is never recorded as a success.	RECOMMENDATION
Relay API and browser capture	A token-based API for Relay and a browser extension, using the same authorization server.	PLANNED
Voice channels	Speech-to-text, spoken replies and voice support. This is an exploratory vision and is not on the current schedule.	FUTURE

7.3 WhatsApp Cloud API: three different things

SCOPE

WhatsApp is not part of Mad3oom's initial launch scope. Nothing in this paper announces it as an available, generally launched customer-facing offering.

Three separate questions are easy to blur, so they are kept apart here.

Table 13. The three layers of the WhatsApp question. The platform's policies change over time and are set by Meta.

QUESTION	WHAT THE EVIDENCE SAYS	STAGE
1. Integration engineering work that exists	A separate WhatsApp module with its own web application and Edge Functions. It connects a business number through Meta's Embedded Signup, verifies Meta's request signature on the webhook (added in September 2026), stores each inbound message once, runs auto-reply flows that stop when a human takes over, and charges template messages against a prepaid wallet. Contract tests cover signature checking, provisioning and the integration API.	IN DEVELOPMENT
2. Technical-provider activities	Meta runs programs for providers that onboard other businesses' accounts, including business verification and app review. Completing them and the commercial terms around them are business and compliance processes that the repositories do not document. This paper therefore states no provider status, partner agreement or approval.	—
3. Future commercial availability	Any general availability would be announced separately and would depend on completing those requirements and on hardening work identified by internal review: per-customer sending credentials, reservation-based wallet billing, and moving WhatsApp onto the shared channel and engine layer.	PLANNED

7.4 What is not claimed

This paper does not claim any partner agreement, customer deployment or integration beyond those in the table above, and it publishes no endpoint addresses or API contracts. Where an integration is listed as planned or in development, the platform does not yet connect that way.

08

CHAPTER 8

Technical Architecture

Mad3oom is built from deliberately plain parts: web pages without a framework, a managed Postgres platform, and a decision engine without dependencies. This chapter states what is verified, what principles shape it, and what is merely recommended.

8.1 Verified technical context

Table 14. Technical context as documented in the repositories on 9 October 2026.

LAYER	WHAT THE REPOSITORIES SHOW
Frontend	HTML, CSS and JavaScript ES modules loaded directly by the browser, with no bundler and no framework; about 90 pages; an Arabic-first, right-to-left interface with a language switch.
Delivery	Vercel serves the static files. Its configuration holds only redirects and rewrites, including those used for OAuth and MCP discovery.
Backend services	Supabase: Postgres with row-level security, Auth, Edge Functions in Deno (about 55 in the platform repository), Realtime, and pg_cron and pg_net for scheduled and asynchronous work.
Business rules	Concentrated in Postgres. SECURITY DEFINER functions check the caller's identity, and the newest tables have no direct table access: they are reached only through such functions.
Intelligence	SIE lives in its own repository as ES modules with no npm dependencies and runs inside Edge Functions. An AI gateway with a multi-provider registry serves other AI features.
Repositories	Three: the platform, the SIE engine, and the WhatsApp module.
Quality tooling	Continuous integration runs tests, a drift check against an inventory of production, and code scanning. The platform has 57 Node test files and 49 SQL test files that run on a real Postgres built from a production-shaped schema snapshot, plus browser render tests; the SIE repository has 1,264 passing tests.

8.2 Conceptual architecture

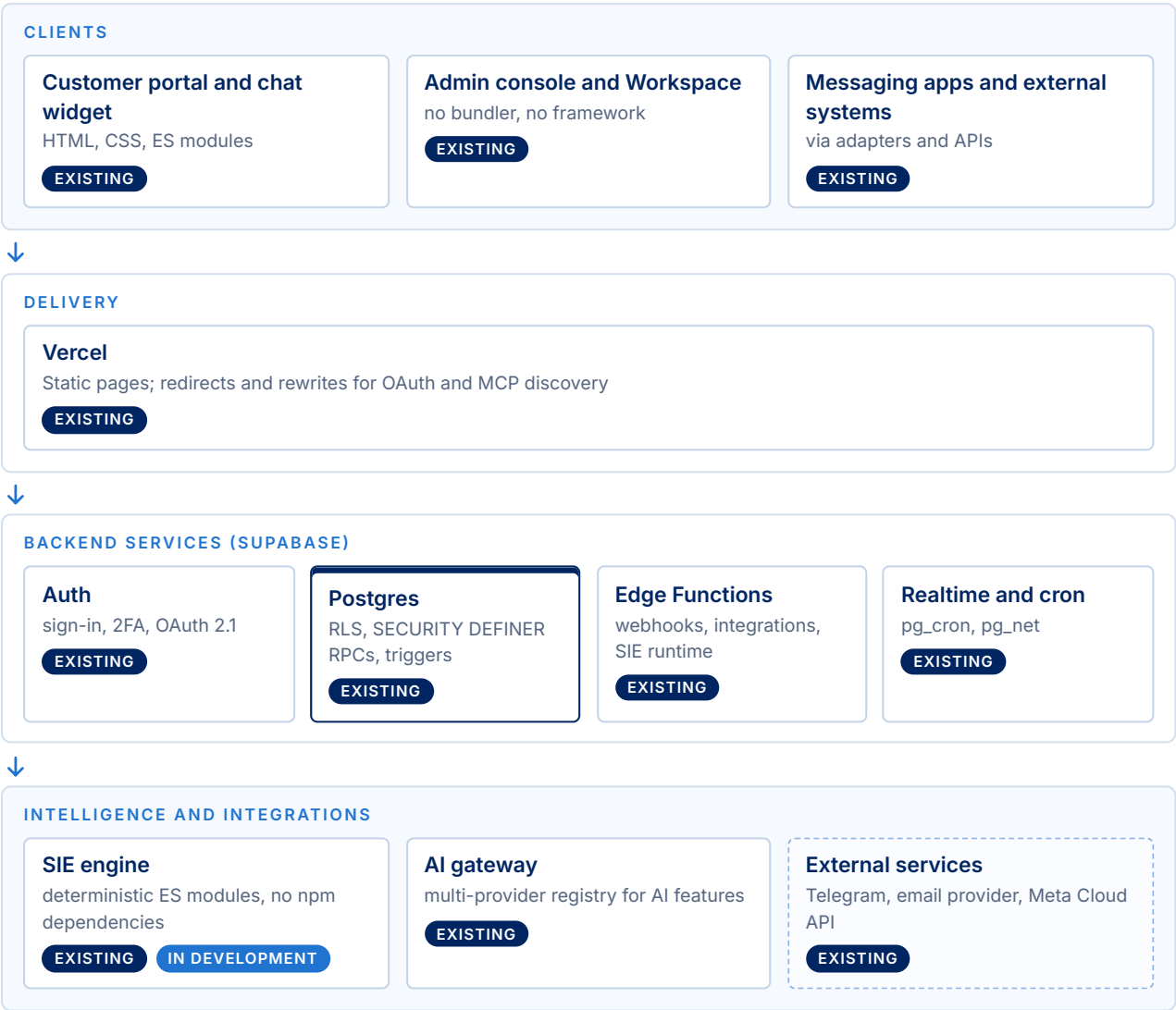


Figure 7. A conceptual overview of the layers. It is not a deployment, network or data-flow diagram, and it omits services that are not documented well enough to draw.

8.3 Principles at work

Table 15. Architectural principles and where they show in the code.

PRINCIPLE	HOW IT APPEARS TODAY	STAGE
Modularity	Frontend features are split into pure model modules, data modules and views. The engine is layered with provider interfaces. Workspace panels are registered, not hard-wired.	EXISTING
Clear interfaces	Database functions are the write boundary; channel adapters have a four-duty contract; the integration API is versioned; the Workspace bridge has a message protocol.	EXISTING
Separation of responsibilities	In SIE, deciding and writing are separate layers and only one layer writes. Workspace never writes: hosted pages do. Domain rules live in the database, not in the doors that reach it.	EXISTING
Maintainability and safe change	Migrations are additive and re-runnable with precondition checks, and recent ones carry rollback scripts. Drift is checked against production. The newest domains use optimistic concurrency and idempotency keys.	EXISTING
Reproducible decisions	The engine has no randomness, injects time and orders ties explicitly, so any decision can be replayed.	EXISTING
One conversation core	A design for idempotent message intake and leased, fenced delivery has been written and tested on a production-shaped database. Its infrastructure was installed in production on 8 October 2026 with every feature flag closed, so no conversation flows through it yet.	IN DEVELOPMENT

8.4 Recommendations and future directions

The items below come from internal engineering reviews in September and October 2026. They are recommendations. None of them is a statement about the current system, and none is scheduled.

Table 16. Recommended architectural improvements.

RECOMMENDATION	WHY	TYPE
Bring the full database baseline under version control and add a staging environment.	Core tables and many functions were created outside the migration history, so a database cannot be rebuilt from the repository alone.	RECOMMENDATION
Pin and self-host front-end dependencies; add a content security policy and integrity checks.	Unpinned remote scripts and the absence of a content policy widen the impact of any upstream change or compromise.	RECOMMENDATION
Replace fire-and-forget side effects in database triggers with an outbox, workers, retries and dead-letter handling.	A failed email or webhook should be visible and retryable, never recorded as a success.	RECOMMENDATION
Make the conversation core the single path for conversations.	It removes parallel code paths for the same message and gives one place for ordering, idempotency and delivery.	RECOMMENDATION

RECOMMENDATION	WHY	TYPE
Propagate the caller's identity into database rules for every API door.	The planned Relay API is designed this way, so one rulebook governs the interface and the API. Applying it elsewhere avoids authorization logic in application code.	RECOMMENDATION
Consolidate duplicated shared code across Edge Functions and keep a single canonical engine deployment path.	Copies drift apart, and two engine versions in production at once make behavior hard to reason about.	RECOMMENDATION
Add request identifiers, error tracking and synthetic checks.	A system can appear healthy while a function fails; operability needs to be designed in.	RECOMMENDATION
Separate unrelated products that share one database.	A fault or compromise in one product should not reach the data of the others.	RECOMMENDATION

LIMITATIONS

This chapter publishes no complete system diagram, no undocumented services, no deployed infrastructure inventory and no API contracts. Where a detailed architecture cannot be verified from the repositories, the paper stays at the conceptual level.

09

CHAPTER 9

Security, Privacy and Governance

A customer-support platform holds conversations, identities and operational history. This chapter lists the principles such a platform should follow, states which safeguards are implemented today, and names the gaps and future work. The principles are design requirements; they are not automatically verified features.

COMPLIANCE STATEMENT

Mad3oom makes no claim of compliance with, or certification against, GDPR, ISO 27001, SOC 2 or any other standard, because none has been independently assessed. No independent penetration test is documented in the repositories. This chapter also omits credentials, tokens, customer data and any detail that could help an attacker.

9.1 Principles, safeguards and gaps

For each principle the table separates what is implemented today in the repositories from the gaps and the work that follows. "Implemented" means present in the code and covered by tests where noted. It does not mean externally audited.

Table 17. Security principles against the implemented safeguards and the gaps.

PRINCIPLE	IMPLEMENTED TODAY	GAPS AND FUTURE WORK
Authentication and authorization	Managed sign-in with several methods and an optional time-based second factor. Authorization rules live in the database: functions check the caller's identity, and page guards only control what is shown. EXISTING	Verify the second factor wherever sensitive data is reached, not only at sign-in; enable leaked-password checks; confirm bot protection on sign-in forms, whose integration exists in code but whose production configuration is not documented. RECOMMENDATION
Role-based and least-privilege access	Separate authority domains for platform staff, company roles and the platform owner. Agents see assigned or team conversations and supervisors see all. In Relay, assigning work to others is an explicit grant. Newer tables have no direct access and are reached only through checked functions. EXISTING	Continue reviewing broadly executable database functions against least privilege. RECOMMENDATION Owner-only erase and privilege grants in Relay are written, not applied. IN DEVELOPMENT

PRINCIPLE	IMPLEMENTED TODAY	GAPS AND FUTURE WORK
Tenant and customer data isolation	Row-level security on tables, a restrictive account-active gate on newer ones, company relations, and an explicit workspace boundary in Relay. SQL tests exercise isolation, including concurrent sessions. EXISTING	Some configuration, such as bot settings, webhooks and automation, is platform-wide rather than per tenant, and several products share one database. Per-tenant configuration and product separation are recommended. RECOMMENDATION
Secure API access	OAuth tokens and API keys are stored hashed, and integration keys are shown once, at creation. OAuth 2.1 uses mandatory PKCE, scoped tokens, rotating refresh tokens and user revocation. Telegram webhooks are checked with a constant-time comparison and are refused if no secret is set; WhatsApp webhooks verify Meta's signature. Token use is rate-limited. EXISTING	Detect reuse of refresh tokens and make their rotation atomic before wider client release; make the inbound-mail webhook fail closed and replay-resistant. RECOMMENDATION
Secret management	Secrets come from the environment, and a test fails if a token-shaped value is committed in the channel code. Integration tokens used by the WhatsApp request proxy are encrypted at rest. EXISTING	Move any remaining legacy plain-text credential storage to encryption or a vault, and never return secrets to the browser. RECOMMENDATION
Auditability of sensitive operations	Relay history is append-only, and a trigger rejects edits and deletions. Inbox actions are logged. Each paid SIE turn is traced with the decision's intent and outcome kept apart. EXISTING	Attribute automated actions to a dedicated system actor rather than to an administrator account. RECOMMENDATION
Data minimization, retention and deletion	Relay stores only selected excerpts, hides them 365 days after closure, redacts them irreversibly with a daily job, and supports deletion requests. A scheduled job archives old tickets. A deletion-request process is published. EXISTING	Chat messages have no retention policy today, and retention for engine traces is not documented. Policies and automated deletion workflows are needed. PLANNED
Protection of conversation content and customer information	Access rules decide who can read a conversation; Relay re-checks access to the original conversation on every excerpt read and fails closed. SIE's trust boundary treats customer text as untrusted data. Attachment paths are validated by a database trigger. EXISTING	Add a content security policy and reduce reliance on manual escaping in the front end. RECOMMENDATION

9.2 Governance and assurance

How change is governed

- Database changes are numbered migrations, written to be re-run safely, with a rollback script for recent ones and an automatic check of the preconditions they rely on.
- Production changes are applied only after explicit approval from the platform owner, which the engineering notes record case by case. Merging code is not treated as approval to change production.
- Read-only verification precedes each application, and a daily drift check compares the repository's inventory with production.

- Engine deployments pin an exact commit, and behavioral changes ship behind flags that default to current behavior.

What assurance exists

- Internal engineering audits in September and October 2026 covered the platform, the nine SIE layers and the conversation core. Their findings drive the recommendations in this paper.
- Automated checks include code scanning, SQL tests for access rules on a real Postgres, and browser tests.
- There is no external penetration test, third-party certification or independent audit documented. None is claimed.

10

CHAPTER 10

Product Differentiation and Value

Mad3oom's intended differentiation is not a single feature. It comes from combining five design principles in one platform. They are presented here as design intent and intended value; none is presented as a proven result.

10.1 Five design principles

Table 18. Differentiation as design principles, with what exists and what would validate each.

PRINCIPLE	INTENDED VALUE	WHAT EXISTS	WHAT WOULD VALIDATE IT
Structured support management	One structured set of records for tickets, conversations and customers, so context is rebuilt less often.	The core platform. EXISTING	Time to reach full context on a case, measured in real use.
Multi-context work through Workspace	Several contexts open at once, with unsent work protected.	Workspace with local layouts. EXISTING	Observed task time and the rate of lost drafts compared with page-by-page work.
Continuity and follow-up through Relay	Commitments that carry evidence, an owner and a deadline, so that fewer requests are dropped.	Core records and ownership rules. EXISTING Handover and reminders. PLANNED	The instrumented measures listed below.
Specialized support intelligence through SIE	Predictable, explainable and testable behavior that stays inside a closed set of actions.	The core engine. EXISTING Remediation under way. IN DEVELOPMENT	Resolution and hand-off quality measured on real conversations, with a defined method.
A modular foundation for integrations	New channels and tools can be added without changing the engine.	The adapter layer, with website and Telegram. EXISTING	The effort needed to bring a second messaging channel into production.

10.2 How the parts combine: an illustrative journey

The value of combining the parts is easiest to see along one illustrative case. Steps marked as planned or future do not yet work this way.

Table 19. An illustrative journey. It shows intent, not a recorded customer case.

STEP	WHAT HAPPENS	PART	STAGE
1	A customer describes a problem in the website chat or on Telegram.	Channels	EXISTING
2	SIE interprets the message, answers from knowledge or asks for detail, and opens a ticket if that is its decision. A human takeover stops it.	SIE	EXISTING
3	An agent opens the conversation, the ticket and the customer's history side by side.	Workspace	EXISTING
4	The agent selects the key messages and records a follow-up with an owner, a next action and a deadline.	Relay	EXISTING
5	If the deadline passes, the record shows as overdue in the list.	Relay	EXISTING
6	A reminder is sent before the deadline, and the work is escalated if nobody acts.	Relay	PLANNED
7	At the end of a shift the agent hands the record over, and the colleague must accept it with its context.	Relay	PLANNED
8	The engine hands a follow-up it cannot settle directly to Relay.	SIE, Relay	FUTURE

10.3 Measures that would show the value

Relay's engineering plan defines the measures below to be instrumented from the first day of use. They are defined and not yet reported, so this paper states no figures for them.

- Records created, by type of source.
- The share of active records that have both an owner and a next action.
- The number and age of overdue records.
- The time a handover waits before it is accepted.
- Reminders delivered compared with reminders that failed.

10.4 What is not claimed

LIMITATIONS

Mad3oom is not compared with named competitors here, and no claim of superiority is made. Differentiation is a statement about how the platform is designed. Whether that design produces better outcomes for a given support team is a question for measurement in real use.

11

CHAPTER 11

Product Roadmap

The roadmap is organized by how real each item is today, not by date. It separates what exists, what is being built, what is planned and what remains a long-term opportunity.

READ THIS FIRST

Roadmap items are subject to technical validation and to business priorities. They are not commitments. No delivery dates, customer milestones or completion claims are given, and the order of items within a category is not a schedule.

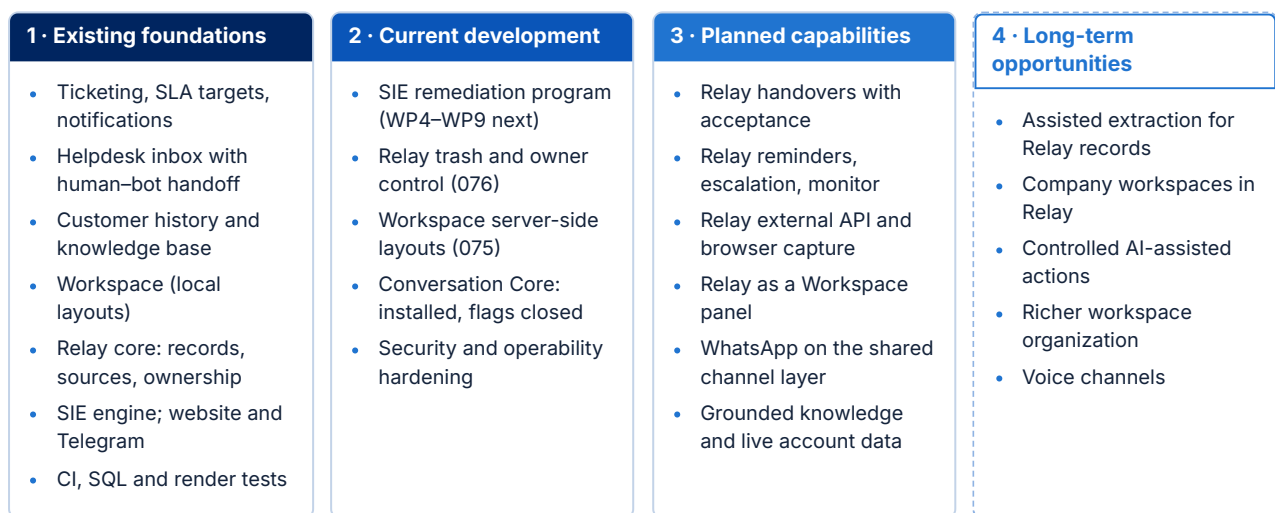


Figure 8. The four roadmap categories at a glance.

11.1 Existing foundations

EXISTING These are implemented in the repositories. They are foundations to build on, not a claim of production maturity.

AREA	FOUNDATION
Support operations	Tickets with SLA targets and notifications; a helpdesk inbox with teams, assignment, tags, notes, scheduled replies and human–bot handoff; customer history; knowledge base and help center; company accounts, roles and an automation builder.
Workspace	Tabs, groups and splits over six panel types, quick open, unsent-work protection, local layout persistence, keyboard and right-to-left support.
Relay	Continuity records, source evidence with access re-checks, ownership and assignment rules, retention and redaction, an audit log, and the native creation flow from the inbox.
SIE	The nine-layer engine, the trust boundary, candidate retrieval, one trace per paid turn, and the website and Telegram channels.
Integration	The channel adapter layer, MCP with OAuth 2.1, API tokens and an integration API.
Engineering	Continuous integration, SQL access-rule tests on a real Postgres, browser tests, drift checks and rollback scripts.

11.2 Current development and refinement

IN DEVELOPMENT Active work, written or partly delivered but not yet complete or enabled for users.

ITEM	DETAIL
SIE remediation program	Delivered: the test foundation, truthful tracing, and language primitives. Sequenced next: the decision layer owning every route, with expiring prompts; evidence that can contradict and isolation between conversations; grounding in live account data and published knowledge; attachment metadata; quota charged only for committed replies and a database-enforced publish gate; and consolidation of customer-visible text.
Relay trash and owner control	Written and tested locally; not applied to production.
Workspace server-side layouts	Written and tested locally; not applied to production. Until then layouts are saved per browser.
Conversation Core	Installed in production with every flag closed. The orchestrator and the engine contract are still to be built before any conversation uses it.
WhatsApp module	Separate module with its own flows and billing logic; outside the initial launch scope.

11.3 Planned capabilities

PLANNED Designed and intended, with no delivered implementation yet.

DIRECTION	ITEMS
Relay	Handovers that must be accepted; reminders, escalation and a continuity monitor; an external API and browser capture; Relay as a Workspace panel.
Expanded integrations	WhatsApp, Messenger and an API channel brought onto the shared channel layer; further tool connections through MCP.
Knowledge workflows	Answers grounded in live account data and published knowledge; review and validation tools wired to publishing.
Workspace	Additional panel types such as knowledge and reports.
Data governance	Retention policies and automated deletion workflows for chat history and engine traces.

11.4 Long-term opportunities

FUTURE Exploratory directions. No commitment is made and none is scheduled.

OPPORTUNITY	WHAT IT WOULD MEAN
Assisted extraction for Relay	Suggested titles, dates and next actions, validated against a strict schema and never confirmed automatically.
Company workspaces in Relay	Relay for companies' own support teams; the data model already carries the workspace boundary.
Controlled AI-assisted actions	Letting the engine propose or perform approved actions through standard tool interfaces, under policy and with human confirmation where needed.
Richer workspace organization	Named layouts for recurring tasks and context shared across panels.
Per-company bots and knowledge	Configuration and knowledge owned by each company, with cited sources.
Shift-aware automation	Automatic handovers once a shift schedule exists as a source in the platform.
Voice channels	Speech-to-text, spoken replies and a voice assistant for agents. An exploratory vision outside the current schedule.

12

CHAPTER 12

Conclusion

Mad3oom's mission is to make technical support more organized, more context-aware, more accountable and more intelligent.

This paper has described how the platform pursues that mission. It brings tickets, conversations, customer records and knowledge into one structured environment. Workspace lets agents work across several contexts without losing what they are doing. Relay keeps follow-up work attached to its evidence, its owner and its deadline. SIE, a layered and explainable engine, interprets what customers write and decides the next step from a closed set of actions.

What exists today is a foundation, not a finished product. The core support features, Workspace, the first parts of Relay and the SIE engine exist in code and are tested. Handovers, reminders, grounded answers, further channels and external APIs are still to come, and the platform has not launched. The paper has kept those two statements side by side, because a credible direction depends on being exact about the starting point.

The direction is consistent. Structure comes first, then context, then accountability, then intelligence, each added in a controlled way. Internal reviews favor validating the design in a controlled pilot before open availability, and they set out hardening work in security and operability that should accompany every new capability. This paper reports no result that has not been measured, and later editions should hold to the same rule.

No business outcome is guaranteed. What the platform can credibly offer is a disciplined architecture, honest reporting of its own status, and steady progress along the roadmap.

A

APPENDIX A

Capability Status Register

One table that gathers the stage of every capability named in this paper, so that a reader can check a claim in one place. The status reflects repository evidence on 9 October 2026.

Table 20. Capability register. Statuses follow the definitions on page 2.

AREA	CAPABILITY	STAGE	CHAPTER
Core platform	Ticket lifecycle, SLA targets, notifications	EXISTING	3
Core platform	Helpdesk inbox: teams, assignment, tags, notes, scheduled replies, human–bot handoff	EXISTING	3
Core platform	Customer history; knowledge base and help center	EXISTING	3
Core platform	Company accounts, roles, automation builder	EXISTING	3
Workspace	Tabs, groups, splits and quick open over six panel types; unsent-work protection	EXISTING	4
Workspace	Layouts saved per browser	EXISTING	4
Workspace	Layouts saved per agent on the server	IN DEVELOPMENT	4
Workspace	Relay and further panel types	PLANNED	4
Workspace	Adaptive, named layouts and shared context	FUTURE	4
Relay	Follow-up and issue records, lifecycle, categories, creation from inbox messages	EXISTING	5
Relay	Ownership and assignment rules; derived overdue flag; audit events	EXISTING	5
Relay	Evidence governance: access re-checks, retention, irreversible redaction	EXISTING	5
Relay	Trash with restore; owner-only erase and grants	IN DEVELOPMENT	5
Relay	Handovers with acceptance, clarification and decline	PLANNED	5
Relay	Reminders, escalation, continuity monitor	PLANNED	5
Relay	External API; browser capture	PLANNED	5
Relay	Assisted extraction; company workspaces	FUTURE	5
SIE	Nine layers: language, scenarios, diagnostics, ranking, decision, dialogue, knowledge, action, observability	EXISTING	6

AREA	CAPABILITY	STAGE	CHAPTER
SIE	Trust boundary; candidate retrieval; sparse session state	EXISTING	6
SIE	Remediation work packages 1–3: test foundation, truthful tracing, language primitives	EXISTING	6
SIE	Remediation work packages 4–9: route ownership, fallible belief, grounding, attachments, metering, dialogue	IN DEVELOPMENT	6
SIE	Grounding in live account data and published knowledge	IN DEVELOPMENT	6
SIE	Handing follow-up work to Relay	FUTURE	6
SIE	Controlled generative or tool-using assistance	FUTURE	6, 11
Integrations	Channel adapter layer; website chat and Telegram	EXISTING	7
Integrations	MCP, OAuth 2.1, API tokens, integration API	EXISTING	7
Integrations	Trigger- and schedule-driven side effects	EXISTING	7
Integrations	WhatsApp module (separate; outside the initial launch scope)	IN DEVELOPMENT	7
Integrations	WhatsApp, Messenger and API channel on the shared layer	PLANNED	7
Integrations	Outbox with retries for side effects	RECOMMENDATION	7, 8
Integrations	Voice channels	FUTURE	7
Architecture	Row-level security, database-function rules, CI, drift checks, rollback scripts	EXISTING	8, 9
Architecture	Conversation Core, installed with flags closed	IN DEVELOPMENT	8
Architecture	Baseline schema in version control, staging, dependency pinning, content policy	RECOMMENDATION	8
Security	Second-factor assurance at the data layer; stronger secret storage; system actor for automation	RECOMMENDATION	9
Security	Retention policies and deletion workflows for chat history and engine traces	PLANNED	9

B

APPENDIX B

Glossary

TERM	MEANING IN THIS PAPER
Action layer	The SIE layer that is the only writer: it commits a decision's message, state and ticket in one atomic transaction.
Adapter (channel)	A module that hides one messaging vendor's details behind a common contract: verify, parse, send.
Arabizi	Arabic written in Latin letters and digits, as often used in chat.
Continuity record	Relay's unit of work: evidence, an owner, a next action, a deadline and an append-only history.
Conversation Core	A database design for idempotent message intake and fenced delivery; installed in production with its flags closed.
Deterministic	Producing the same output for the same input and configuration; no randomness, injected time, explicit tie-breaking.
Drift check	An automated comparison between the repository's inventory of database objects and production.
Edge Function	Server-side code run by Supabase close to the user, used for webhooks, integrations and the SIE runtime.
Embed mode	A mode in which an existing page drops its surrounding chrome and shows one record, so Workspace can host it.
Evaluated rules	The record, kept with each SIE decision, of every rule that was checked, whether it matched and why.
Fail closed	When a check cannot be completed, deny or hide rather than allow.
Idempotency key	A client-chosen identifier that makes a retried request return the first result instead of creating a duplicate.
MCP	Model Context Protocol, a standard way for AI clients to use tools and data exposed by a server.
OAuth 2.1 and PKCE	A standard for delegated, scoped authorization; PKCE protects the exchange of authorization codes.
Optimistic concurrency	Rejecting an edit made against a stale version of a record instead of overwriting newer changes.
Panel	One tab in Workspace, hosting a page or a single record.
RLS	Row-level security: database rules that decide which rows a user can read or change.

TERM	MEANING IN THIS PAPER
RPC (SECURITY DEFINER)	A database function called by the client that checks the caller's identity and performs a controlled operation.
Scenario	An authored description of a support situation in SIE's catalog: expected evidence, resolutions and questions.
SIE	Support Intelligence Engine, Mad3oom's deterministic engine for interpreting support conversations.
SLA	Service-level agreement; here, a target time to resolve a ticket by priority.
Trust boundary	The points where customer text crosses from input into something that controls the engine, treated as untrusted.
Work package (WP)	A unit of the SIE remediation program, with stated exit criteria.



APPENDIX C

Basis of Preparation and Limitations

How the statements in this paper were established, and what was not verified. A white paper is only as credible as its evidence.

C.1 Sources reviewed

- The source code and engineering documents of three repositories: the Mad3oom platform, the SIE engine and the WhatsApp module, as they stood on 9 October 2026.
- Architecture and implementation records, including the Workspace and Relay design documents, the SIE architecture, guarantee registry and nine-layer audit, the conversation-core gate and installation notes, and the October 2026 engineering audit of the platform.
- Database migrations, rollback scripts and the automated tests in each repository.

C.2 Method

- Statements about what exists rest on the code and on the engineering records. Statements about production rest on those records, which note what was applied and when.
- The SIE engine's automated suite was run on the repository's latest commit while this paper was prepared, and 1,264 tests passed.
- Figures from engineering records, such as test counts, code sizes and timings, are quoted with their date and with what they measure.
- The English and Arabic editions are produced from one source, so that their structure and facts match. The Arabic edition is equivalent in meaning, not a word-for-word translation.

C.3 What was not verified

NOT VERIFIED	CONSEQUENCE FOR THIS PAPER
Production usage and volumes	No customer counts, usage figures or deployments are reported.
Whether each merged change is live for end users	"Existing" means implemented in the repositories, not confirmed live.
Real-world accuracy of SIE	No accuracy, resolution or satisfaction figures are reported.
Security assessment by a third party	No certification or compliance is claimed.
Meta provider status and commercial terms	No partnership or provider status is stated.
Changes after 9 October 2026	They are not reflected in this edition.

C.4 Editorial rules

- Every capability carries a stage label, and a planned feature is never described as a current capability.
- Passing automated tests are never presented as proof of production reliability.
- No competitor is named and no superiority is claimed. No credentials, customer data or exploitable security detail is included.
- Other public material, such as the website's roadmap page last updated in July 2026, is not used as evidence. Where it differs from this paper, this paper follows the repository evidence.

C.5 Document history

VERSION	DATE	NOTE
1.0	October 2026	Foundational edition. Information as of 9 October 2026.